

```
[2]
✓ 17s import tensorflow as tf
      from tensorflow.keras import layers
      import numpy as np
      import matplotlib.pyplot as plt

Double-click (or enter) to edit

[3]
✓ 2s BUFFER_SIZE = 60000
      BATCH_SIZE = 256

      (x_train, _), (_, _) = tf.keras.datasets.mnist.load_data()

      x_train = (x_train.astype("float32") - 127.5) / 127.5
      x_train = np.expand_dims(x_train, axis=-1)

      dataset = tf.data.Dataset.from_tensor_slices(x_train)
      dataset = dataset.shuffle(BUFFER_SIZE).batch(BATCH_SIZE)

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ————— 0s 0us/step

[4]
▶ ✓ 1s def build_generator():
      model = tf.keras.Sequential()

[3]
✓ 2s x_train = (x_train.astype("float32") - 127.5) / 127.5
      x_train = np.expand_dims(x_train, axis=-1)

      dataset = tf.data.Dataset.from_tensor_slices(x_train)
      dataset = dataset.shuffle(BUFFER_SIZE).batch(BATCH_SIZE)

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ————— 0s 0us/step

[4]
▶ ✓ 1s def build_generator():
      model = tf.keras.Sequential()

      model.add(layers.Dense(7 * 7 * 256, input_shape=(100,)))
      model.add(layers.Reshape((7, 7, 256)))
      model.add(layers.BatchNormalization())
      model.add(layers.LeakyReLU())

      model.add(layers.Conv2DTranspose(128, kernel_size=5, strides=1, padding='same'))
      model.add(layers.BatchNormalization())
      model.add(layers.LeakyReLU())

      model.add(layers.Conv2DTranspose(64, kernel_size=5, strides=2, padding='same'))
      model.add(layers.BatchNormalization())
      model.add(layers.LeakyReLU())

      model.add(layers.Conv2DTranspose(1, kernel_size=5, strides=2,
                                      padding='same', activation='tanh'))

      return model

      generator = build_generator()

... /usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an
      super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
[4]
✓ 1s
  /usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass
    super().__init__(activity_regularizer=activity_regularizer, **kwargs)

[5]
✓ 0s
▶ def build_discriminator():
    model = tf.keras.Sequential()

    model.add(layers.Conv2D(64, kernel_size=5, strides=2,
        padding='same', input_shape=(28, 28, 1)))
    model.add(layers.LeakyReLU())
    model.add(layers.Dropout(0.3))

    model.add(layers.Conv2D(128, kernel_size=5, strides=2, padding='same'))
    model.add(layers.LeakyReLU())
    model.add(layers.Dropout(0.3))

    model.add(layers.Flatten())
    model.add(layers.Dense(1, activation='sigmoid'))
    return model

discriminator = build_discriminator()

... /usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarni
    super().__init__(activity_regularizer=activity_regularizer, **kwargs)

[6]
✓ 0s
cross_entropy = tf.keras.losses.BinaryCrossentropy(from_logits=False)

def discriminator_loss(real_output, fake_output):
    real_loss = cross_entropy(tf.ones_like(real_output), real_output)
    fake_loss = cross_entropy(tf.zeros_like(fake_output), fake_output)
    return real_loss + fake_loss

def generator_loss(fake_output):
    return cross_entropy(tf.ones_like(fake_output), fake_output)
```

```
[6] ✓ 0s cross_entropy = tf.keras.losses.BinaryCrossentropy(from_logits=False)

def discriminator_loss(real_output, fake_output):
    real_loss = cross_entropy(tf.ones_like(real_output), real_output)
    fake_loss = cross_entropy(tf.zeros_like(fake_output), fake_output)
    return real_loss + fake_loss

def generator_loss(fake_output):
    return cross_entropy(tf.ones_like(fake_output), fake_output)

[7] ✓ 0s generator_optimizer = tf.keras.optimizers.Adam(learning_rate=0.0002, beta_1=0.5)
discriminator_optimizer = tf.keras.optimizers.Adam(learning_rate=0.0002, beta_1=0.5)

[8] ✓ 0s @tf.function
def train_step(images):
    noise = tf.random.normal([BATCH_SIZE, 100])

    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:
        fake_images = generator(noise, training=True)

        real_output = discriminator(images, training=True)
        fake_output = discriminator(fake_images, training=True)

        gen_loss = generator_loss(fake_output)
        disc_loss = discriminator_loss(real_output, fake_output)

        gradients_gen = gen_tape.gradient(gen_loss, generator.trainable_variables)
        gradients_disc = disc_tape.gradient(disc_loss, discriminator.trainable_variables)

        generator_optimizer.apply_gradients(zip(gradients_gen, generator.trainable_variables))
        discriminator_optimizer.apply_gradients(zip(gradients_disc, discriminator.trainable_variables))

    return gen_loss, disc_loss

[9] ✓ 21s EPOCHS = 1
seed = tf.random.normal([16, 100])

def generate_and_save_images(model, epoch, test_input):
    predictions = model(test_input, training=False)
    predictions = (predictions + 1) / 2.0 # back to [0,1]

    fig = plt.figure(figsize=(4, 4))
    for i in range(predictions.shape[0]):
        plt.subplot(4, 4, i + 1)
        plt.imshow(predictions[i, :, :, 0], cmap='gray')
        plt.axis('off')

    plt.suptitle(f"Epoch {epoch}")
    plt.show()

    for epoch in range(EPOCHS):
        for image_batch in dataset:
            gen_loss, disc_loss = train_step(image_batch)

        if (epoch + 1) % 5 == 0:
            generate_and_save_images(generator, epoch + 1, seed)
            print(f"Epoch {epoch+1}, Gen Loss: {gen_loss:.4f}, Disc Loss: {disc_loss:.4f}")

[10] ✓ 0s plt.figure(figsize=(4,4))
for i in range(16):
    plt.subplot(4,4,i+1)
    plt.imshow(x_train[i,:,:,:], cmap='gray')
    plt.axis('off')
plt.suptitle("Real MNIST Images")
```

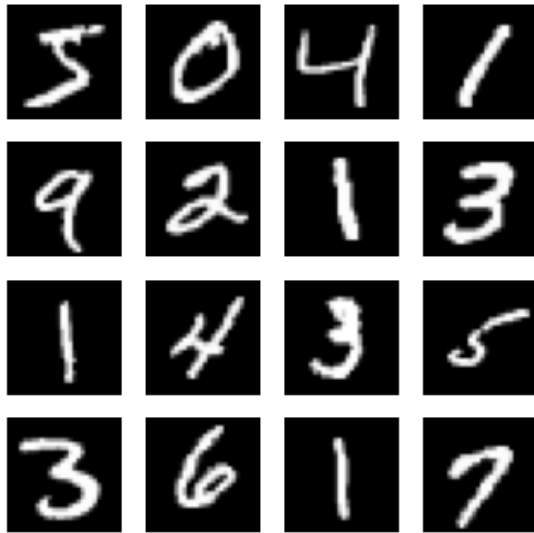


```
plt.figure(figsize=(4,4))
for i in range(16):
    plt.subplot(4,4,i+1)
    plt.imshow(x_train[i,:,:,:0], cmap='gray')
    plt.axis('off')
plt.suptitle("Real MNIST Images")
plt.show()

generate_and_save_images(generator, "Final", seed)
```

...

Real MNIST Images



Epoch Final



Epoch Final

